UNIVERSIDAD DOCTOR ANDRÉS BELLO

FACULTAD DE TECNOLOGÍA E INNOVACIÓN

INGENIERÍA EN SISTEMAS

DISEÑO DE SEGURIDAD DE REDES

TERCER LABORATORIO

VERIFICACIÓN DE AVANCES DEL PROYECTO

BlindNote

Informe del proyecto y verificación del checklist

Implementación integral de medidas de seguridad informática en sitios web

Integrantes

Nombre	Rol / tarea
_____	_____
_____	_____
_____	_____
_____	_____

Docente: _____

Octubre de 2026

Versión 1.0

1. Resumen ejecutivo

BlindNote es una aplicación web académica desarrollada en PHP 8 y MySQL para demostrar, de forma práctica, la implementación de medidas de seguridad en un sitio web. El proyecto incluye autenticación con contraseñas cifradas, control de acceso basado en roles, protección contra inyección SQL, XSS y CSRF, bloqueo por fuerza bruta, auditoría de eventos, cabeceras de seguridad HTTP y una política de seguridad documentada.

La aplicación se encuentra operativa y fue verificada en el entorno local. El checklist del Tercer Laboratorio se cumple en **4 de los 6 aspectos** de forma completa. Los dos aspectos restantes dependen de evidencias externas: la demostración de HTTPS en el servidor configurado y la tabla de participación individual de los integrantes. Con esas dos evidencias, el checklist queda completo.

Resultado	Detalle
Criterios completos	1, 3, 4 y 5
Criterios pendientes de evidencia	2 (HTTPS en servidor) y 6 (participación individual)
Puntaje verificado en local	7 / 10
Puntaje potencial con evidencias	10 / 10

2. Introducción

La seguridad de las aplicaciones web no depende de una sola medida, sino de un conjunto de controles que actúan en capas: validación de entradas, cifrado de credenciales, control de sesiones, autorización, protección de formularios, registro de actividad y configuración segura del servidor.

El proyecto BlindNote fue construido para aplicar esos controles sobre un caso realista y verificable: un portal con usuarios, roles, notas personales y un panel administrativo. Cada medida implementada puede probarse desde el navegador y está respaldada por código y por registros de auditoría.

3. Objetivos

3.1 Objetivo general

Implementar un sitio web seguro que demuestre la aplicación de controles de seguridad informática en autenticación, control de acceso, protección de datos y análisis de vulnerabilidades.

3.2 Objetivos específicos

- Implementar un formulario de inicio de sesión con validación en servidor.
- Almacenar las contraseñas con un algoritmo de hash seguro (bcrypt).
- Aplicar control de acceso basado en roles y verificación de propiedad.
- Mitigar inyección SQL, XSS, CSRF y fijación de sesión.
- Incorporar bloqueo por fuerza bruta, auditoría y cabeceras de seguridad.
- Documentar una política de seguridad y un procedimiento de respuesta a incidentes.
- Verificar el cumplimiento del checklist del Tercer Laboratorio.

4. Alcance

El proyecto cubre:

- Aplicación web con módulos de login, registro, panel, notas, administración e informe de seguridad.
- Base de datos MySQL con usuarios, notas, intentos de login y auditoría.
- Configuración de seguridad de la aplicación y del servidor web.
- Documentación de políticas y pruebas.

No cubre la infraestructura de red de la universidad ni un despliegue de producción con datos reales. El sitio es de uso académico y debe ejecutarse en un entorno controlado.

5. Descripción general del proyecto

BlindNote es un portal que permite:

1. Registrarse con una contraseña que cumple una política mínima.

2. Iniciar sesión con usuario o correo y contraseña.

3. Acceder a un panel con el resumen de la cuenta y las medidas activas.

4. Crear y eliminar notas personales.

5. Administrar usuarios y revisar la auditoría (solo rol `admin`).

6. Consultar el informe de la Fase 4 con herramientas, hallazgos y soluciones.

La aplicación aplica el principio de **defensa en profundidad**: aunque un atacante fuerce una URL o modifique un formulario, el servidor vuelve a validar la sesión, el rol, la propiedad del recurso y el token CSRF.

6. Arquitectura y tecnologías

Componente	Tecnología	Función
Servidor web	Apache 2.4 (Laragon) / Nginx	Publica el sitio y aplica `.htaccess` o configuración equivalente.
Lenguaje	PHP 8.1	Lógica de la aplicación y controles de seguridad.
Base de datos	MySQL 8 / MariaDB	Almacena usuarios, notas, intentos y auditoría.
Acceso a datos	PDO con consultas preparadas	Evita inyección SQL.
Hash de contraseñas	bcrypt, coste 12	Almacenamiento irreversible de contraseñas.
Sesiones	PHP Sessions endurecidas	Control de autenticación y expiración.
Frontend	HTML5 + CSS propio	Interfaz responsive sin dependencias externas.
HTTPS	Certificado en servidor de despliegue	Cifrado del tráfico y cookies `Secure`.

6.1 Capas de la aplicación

1. **Capa de presentación:** páginas PHP con HTML y CSS (`login.php`, `dashboard.php`, `notas.php`, `admin.php`).
2. **Capa de lógica:** funciones de autenticación, autorización, CSRF, auditoría y validación (`includes/auth.php`, `includes/functions.php`).
3. **Capa de datos:** conexión PDO y consultas preparadas (`config/database.php`, `database/schema.sql`).
4. **Capa de configuración:** parámetros de seguridad y conexión (`config/config.php`, `.htaccess`).

6.2 Estructura de la base de datos

Tabla	Contenido	Uso de seguridad
`usuarios`	Cuentas, rol, estado, hash bcrypt, último login.	Autenticación y autorización.
`notas`	Notas personales de cada usuario.	Verificación de propiedad (anti-IDOR).
`intentos_login`	Intentos exitosos y fallidos por usuario/IP.	Control de fuerza bruta.
`logs_auditoria`	Acciones sensibles y accesos denegados.	Trazabilidad y respuesta a incidentes.

7. Funcionalidades principales

7.1 Registro de usuarios

- Valida nombre, correo, usuario y contraseña en el servidor.
- Exige mínimo 8 caracteres, mayúscula, minúscula y número.
- Comprueba duplicados con consultas preparadas.
- Cifra la contraseña con `password_hash()` y bcrypt coste 12.
- El primer usuario registrado obtiene el rol `admin`; los siguientes, `usuario`.

7.2 Inicio de sesión

- Acepta usuario o correo electrónico.
- Valida el token CSRF antes de procesar.
- Comprueba credenciales con `password_verify()`.
- Aplica bloqueo tras 5 intentos fallidos en 15 minutos.
- Usa mensajes genéricos para no revelar si la cuenta existe.
- Regenera el identificador de sesión al autenticar.
- Registra el intento y el evento en la auditoría.

7.3 Panel del usuario

- Muestra el total de notas, el último inicio de sesión y la IP del cliente.
- Presenta un resumen de las medidas de seguridad activas.
- Verifica que el User-Agent de la sesión no haya cambiado.

7.4 Notas personales

- Crear y eliminar notas.
- Cada consulta incluye el identificador del usuario autenticado.
- El contenido se escapa con `htmlspecialchars()` antes de mostrarse.
- Las operaciones de escritura exigen token CSRF.

7.5 Administración

- Disponible solo para el rol `admin`.
- Lista usuarios, roles, estado y último acceso.
- Muestra los últimos eventos de auditoría.
- Si un usuario normal intenta entrar, recibe HTTP 403 y queda registrado.

7.6 Cierre de sesión

- Solo se acepta por método POST.
- Exige token CSRF.
- Destruye la sesión y elimina la cookie del navegador.

8. Medidas de seguridad implementadas

Medida	Implementación	Amenaza mitigada
Hash de contraseñas	bcrypt coste 12 con `password_hash()` / `password_verify()`	Robo de credenciales en la base de datos.
Consultas preparadas	PDO con `ATTR_EMULATE_PREPARES = false`	Inyección SQL.
Escape de salida	`htmlspecialchars()` mediante `e()`	XSS reflejado y almacenado.
Token CSRF	Token aleatorio por sesión validado con `hash_equals()`	CSRF en formularios y logout.
Sesiones seguras	`HttpOnly`, `SameSite=Strict`, `Secure` bajo HTTPS, regeneración de ID	Robo y fijación de sesión.
Expiración de sesión	20 minutos de inactividad y 8 horas máximas	Uso de sesiones abandonadas.
Control de acceso	RBAC con `exigir_rol()` y verificación de propiedad	Escalada de privilegios e IDOR.
Fuerza bruta	5 intentos / 15 minutos por usuario e IP	Ataques de diccionario y credential stuffing.
Mensajes genéricos	Error uniforme en login	Enumeración de usuarios.
Auditoría	`logs_auditoria` e `intentos_login`	Falta de trazabilidad.
Cabeceras HTTP	CSP, X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy	Clickjacking, MIME sniffing y fugas de información.
Protección de archivos	`.htaccess` bloquea `.sql`, `.md`, `.json`, `.log` y carpetas internas	Exposición de configuración y respaldos.

8.1 Controles por fase del proyecto

Fase	Controles
Fase 3	Login, registro, bcrypt, sesiones, RBAC, CSRF, fuerza bruta, auditoría.
Fase 4	Consultas preparadas, escape XSS, cabeceras, IDOR, informe de hallazgos.
Fase 5	Política de seguridad, copias de seguridad, respuesta a incidentes.

9. Verificación del checklist del Tercer Laboratorio

La siguiente tabla resume el estado de cada aspecto solicitado en el documento del laboratorio.

#	Aspecto a verificar	Estado	Evidencia en el proyecto
1	Entorno de trabajo: servidor web, carpeta del sitio y página accesible por HTTP.	**Cumple**	Apache 2.4.54, PHP 8.1.10 y MySQL 8 en ejecución. Sitio accesible en `http://seguridad-web.test:88/login.php` con HTTP 200.
2	HTTPS: certificado digital creado y configuración inicial demostrada.	**Pendiente de evidencia**	La aplicación está preparada para HTTPS: `COOKIE_SOLO_HTTPS`, HSTS documentado y pasos en el README. La demostración se realizará en el servidor con certificado configurado.
3	Control de acceso: formulario de login con validación y manejo seguro de contraseñas.	**Cumple**	`login.php`, `register.php`, bcrypt coste 12, CSRF, bloqueo por fuerza bruta, regeneración de sesión y RBAC.
4	Pruebas de seguridad: herramienta seleccionada y primeras pruebas o plan documentado.	**Cumple**	`informe_fase4.php` documenta OWASP ZAP, Nikto, SQLMap y Mozilla Observatory, hallazgos, severidad, soluciones y pasos de reproducción.
5	Política de seguridad: contraseñas/usuarios, copias de seguridad y respuesta ante incidentes.	**Cumple**	`POLITICA_SEGURIDAD.md` contiene los tres componentes exigidos y controles adicionales.

#	Aspecto a verificar	Estado	Evidencia en el proyecto
6	Participación individual: cada integrante explica una tarea y muestra evidencia propia.	**Pendiente de completar**	Se debe llenar la tabla del Anexo B con el nombre, la tarea y la evidencia de cada integrante.

9.1 Resultado del checklist

- **Cumplen:** 1, 3, 4 y 5.
- **Pendientes de evidencia externa:** 2 y 6.
- **Puntaje verificado en el entorno local:** 7 / 10.
- **Puntaje potencial al completar las evidencias:** 10 / 10.

La pendiente de HTTPS no requiere cambios de código: el servidor de despliegue ya contará con el certificado. La pendiente de participación se resuelve con la tabla de integrantes y sus capturas o registros de trabajo.

10. Evidencias de funcionamiento

Las siguientes pruebas se ejecutaron contra el sitio local y confirman el funcionamiento de los controles:

Prueba	Resultado obtenido
Acceso a `login.php` por HTTP	HTTP 200.
Inicio de sesión con credenciales válidas	HTTP 302 hacia `dashboard.php`; sesión creada.
Acceso al panel autenticado	HTTP 200.
POST de login sin token CSRF	HTTP 403; solicitud rechazada.
Login con carga SQLi `admin' OR '1'='1`	Acceso denegado; mensaje genérico.
Usuario con rol `usuario` accede a `admin.php`	HTTP 403; acceso denegado.
Nota con `<script>alert(1)</script>`	Se muestra escapada como texto; no se ejecuta.
Acceso a `setup.php` después de eliminarlo	HTTP 404.
Sintaxis de los archivos PHP	15 archivos verificados sin errores.
Cabeceras de seguridad	CSP, X-Frame-Options, nosniff, Referrer-Policy y Permissions-Policy presentes.
Cookie de sesión	`HttpOnly` y `SameSite=Strict`.

> *La prueba de HTTPS se realizará en el servidor configurado con el certificado digital. En ese entorno se debe verificar el candado del navegador, la URL `https://`, la cookie `Secure` y la cabecera HSTS.*

11. Fase 4 — Análisis de vulnerabilidades

El informe `informe_fase4.php` documenta las herramientas y los hallazgos:

Herramienta	Uso
OWASP ZAP	Escaneo pasivo y activo de la aplicación.
Nikto	Revisión de configuración del servidor y archivos expuestos.
SQLMap	Pruebas de inyección SQL.
Mozilla Observatory	Verificación de cabeceras de seguridad.

Hallazgos principales y estado:

Vulnerabilidad	Severidad	Estado
Inyección SQL	Crítica	Mitigada con PDO preparado.
XSS almacenado	Alta	Mitigada con escape de salida y CSP.
CSRF	Alta	Mitigada con token por sesión.
Fijación de sesión	Alta	Mitigada con regeneración de ID.
Fuerza bruta	Alta	Mitigada con bloqueo temporal y auditoría.
Contraseñas en texto plano	Crítica	Mitigada con bcrypt.
Cabeceras ausentes	Media	Mitigadas con cabeceras HTTP.
IDOR	Alta	Mitigada con verificación de propiedad.
Métodos HTTP inseguros	Baja	Recomendación de configuración del servidor.

Para fortalecer la evidencia, se recomienda ejecutar ZAP o Nikto durante la demostración y adjuntar el reporte o capturas de pantalla.

12. Fase 5 — Política de seguridad

La política completa está en `POLITICA_SEGURIDAD.md` e incluye:

- Objetivo, alcance, principios y roles.
- Gestión de usuarios, altas, bajas, revisión de accesos y roles.
- Política de contraseñas, bcrypt, bloqueo por fuerza bruta y sesiones.
- Copias de seguridad, retención, regla 3-2-1, RPO/RTO y pruebas de restauración.
- Registro, monitoreo y auditoría.
- Respuesta ante incidentes: clasificación, fases, procedimientos y evidencias.
- Configuración segura del servidor, HTTPS y gestión de cambios.
- Anexos con checklist, contactos, referencias e historial de versiones.

13. Conclusiones

1. El proyecto implementa controles de seguridad verificables en todas las capas de la aplicación.

2. Los aspectos 1, 3, 4 y 5 del checklist están completos y sustentados con código, documentación y pruebas.

3. La aplicación demuestra mitigaciones efectivas contra SQLi, XSS, CSRF, IDOR, fuerza bruta y fijación de sesión.

4. La política de seguridad cubre los tres temas exigidos: usuarios/contraseñas, copias de seguridad y respuesta ante incidentes.

5. Para completar el checklist al 100 % solo faltan las evidencias de HTTPS en el servidor de despliegue y la participación individual de los integrantes.

14. Recomendaciones

- Demostrar HTTPS con un certificado válido o autofirmado y capturar la URL, el candado y la cookie `Secure`.
- Ejecutar OWASP ZAP o Nikto y conservar el reporte como evidencia.
- Completar la tabla de integrantes con tareas y capturas propias.
- En producción: deshabilitar el registro público, usar un usuario de base de datos con privilegios mínimos, activar HSTS y programar copias de seguridad probadas.
- Mantener actualizados PHP, Apache/Nginx, MySQL y las dependencias.

Anexo A. Guion de demostración

1. Abrir `http://seguridad-web.test:88/login.php` y mostrar el formulario.

2. Intentar un POST sin token CSRF y mostrar el HTTP 403.

3. Iniciar sesión con una cuenta válida y mostrar el panel.

4. Crear una nota con `<script>alert(1)</script>` y comprobar que se muestra como texto.

5. Intentar acceder a `admin.php` con un usuario normal y mostrar el 403.

6. Entrar como administrador y mostrar usuarios y auditoría.

7. Mostrar las cabeceras de seguridad con las herramientas del navegador o `curl -I`.

8. Abrir `informe_fase4.php` y explicar herramientas y hallazgos.

9. Presentar `POLITICA_SEGURIDAD.md` y resumir sus tres ejes.

10. En el servidor con certificado, abrir `https://` y mostrar el candado, la cookie `Secure` y HSTS.

11. Cerrar sesión y comprobar que la sesión se destruye.

Anexo B. Participación individual

Integrante	Tarea realizada	Evidencia (captura, código o registro)
_____	_____	_____
_____	_____	_____
_____	_____	_____
_____	_____	_____

Anexo C. Referencias

- OWASP Top 10 y OWASP ASVS.
- OWASP ZAP: <https://www.zaproxy.org>.
- Nikto: <https://cirt.net/Nikto2>.
- NIST SP 800-61: respuesta a incidentes.
- ISO/IEC 27001: gestión de seguridad de la información.